

🔍 Linear Search Algorithm in Python

Definition:

Linear Search (also called Sequential Search) is the simplest search algorithm that checks every element in a list one by one until the desired element is found or the list ends.

✔ Characteristics:

Feature	Description
Time Complexity	$O(n)$ — checks each element once
Space Complexity	$O(1)$ — no extra space needed
Best Case	$O(1)$ — if the target is at the beginning
Worst Case	$O(n)$ — if the target is at the end or not found
Data Requirement	Works on unsorted or sorted lists

❑ How It Works:

- Start from index 0.
 - Compare each element with the target.
 - If match found, return index or element.
 - If not found after checking all elements, return -1 or "Not found".
-

❑ Python Example:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i # return the index of the found element
    return -1 # target not found

# Example usage
numbers = [5, 3, 7, 1, 9, 2]
target = 1

result = linear_search(numbers, target)
if result != -1:
    print(f"Element found at index: {result}")
else:
    print("Element not found")

# ❑ Output:
# Element found at index: 3
```

✦ When to Use Linear Search?

- When the dataset is small.
 - When the list is unsorted.
 - When performance is not critical.
-

🔍 What is Binary Search?

Binary Search is an efficient algorithm used to find a specific element in a sorted list. It works by dividing the search space in half again and again, reducing the time it takes to find the item.

✓ Prerequisite

- The list **must be sorted** in ascending order (very important).
 - Binary search is more efficient than linear search, especially for large lists.
-

💡 Idea (Analogy)

Suppose you are looking for a number in a book that contains sorted page numbers. Instead of going page by page, you:

- Open the book in the middle.
- Check if it's the page you're looking for.
- If not, decide whether to go left (lower numbers) or right (higher numbers).
- Repeat until you find the page.

☐ This is what binary search does in code.

🔄 Steps (Algorithm)

1. Start with the `left` and `right` pointers at the start and end of the list.
 2. While `left` is less than or equal to `right`:
 - Calculate the middle index.
 - Compare the target with the middle element:
 - If equal → Found the item.
 - If `target < middle` → Search in the **left half**.
 - If `target > middle` → Search in the **right half**.
 3. If the item is not found, return `-1` or "Not Found".
-

☐ Time & Space Complexity

Case	Complexity
------	------------

Best Case	O(1) — Found in the middle
-----------	-----------------------------------

Worst Case	O(log n) — Keep dividing
------------	---------------------------------

Case	Complexity
Space	O(1) — No extra space

🔗 Python Code with Detailed Comments

```
def binary_search(arr, target):
    # Step 1: Initialize pointers
    left = 0          # Start of the list
    right = len(arr) - 1  # End of the list

    # Step 2: Repeat the loop until left crosses right
    while left <= right:
        # Step 3: Find the middle index
        mid = (left + right) // 2

        # Step 4: Check the value at mid
        if arr[mid] == target:
            return mid  # Found the target

        # Step 5: Decide where to search next
        elif target < arr[mid]:
            right = mid - 1
        else:
            left = mid + 1

    # Step 6: If not found, return -1
    return -1
```

🔗 Example

```
numbers = [2, 4, 6, 8, 10, 12, 14]  # List must be sorted
target = 10

result = binary_search(numbers, target)

if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")
```

Output:

```
Element found at index 4
```

☐ Dry Run (Step-by-Step)

```
arr = [2, 4, 6, 8, 10], target = 8
left = 0, right = 4

mid = (0 + 4) // 2 = 2 → arr[mid] = 6

8 > 6 → search right → left = 3
```

```
mid = (3 + 4) // 2 = 3 → arr[mid] = 8  
8 == 8 → FOUND!
```

⊗ What if the list is unsorted?

```
arr = [10, 4, 6, 2, 8] # Not sorted
```

Binary search will give **wrong results**.

✓ Always sort the list first!

★ Bonus: Recursive Version in Python

```
def binary_search_recursive(arr, target, left, right):  
    if left > right:  
        return -1  
  
    mid = (left + right) // 2  
  
    if arr[mid] == target:  
        return mid  
    elif target < arr[mid]:  
        return binary_search_recursive(arr, target, left, mid - 1)  
    else:  
        return binary_search_recursive(arr, target, mid + 1, right)  
  
# Example  
arr = [1, 3, 5, 7, 9]  
print(binary_search_recursive(arr, 7, 0, len(arr) - 1)) # Output: 3
```

✓ Summary

Step	Action
1	Check if the list is sorted
2	Set left and right pointers
3	Loop until <code>left <= right</code>
4	Check middle value
5	Narrow the search (left/right half)
6	Return index if found or -1 if not

♣ What is DFS (Depth First Search)?

DFS is a graph traversal algorithm that starts from a source node and explores as far as possible along each branch before backtracking.

It uses a **stack data structure** (either explicitly or via recursion) and is commonly used in:

- ☐ Solving mazes
 - ☐ Path finding
 - 🔁 Topological sorting
 - 🔁 Cycle detection
 - ☐ Solving puzzles (e.g., Sudoku)
-

★ Key Concepts

- **Graph:** A collection of nodes (vertices) connected by edges.
 - DFS works on both **directed** and **undirected** graphs.
 - DFS can be implemented using:
 - **Recursion**
 - **Explicit Stack**
-

🔄 DFS Process (Step-by-Step)

1. Start at a node (called source).
 2. Mark the node as visited.
 3. For each unvisited neighbor:
 - Recursively apply DFS.
 4. Backtrack when no unvisited neighbor is left.
-

✓ DFS Pseudocode (Recursive)

```
DFS(node) :  
    if node is not visited:  
        mark node as visited  
        for each neighbor of node:  
            DFS(neighbor)
```

☐ DFS vs BFS (Key Differences)

Feature	DFS	BFS
Data Structure	Stack / Recursion	Queue
Explores	As deep as possible	Level by level
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$ (recursion/stack)	$O(V)$ (queue)
Use Case	Pathfinding, Cycle Detection Shortest path in unweighted graphs	

DFS Code in Python

1 Recursive DFS for Adjacency List Graph

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

# Example graph (undirected)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()
dfs(graph, 'A', visited)
# Output: A B D E F C (order may vary based on graph structure)
```

2 Iterative DFS Using Stack

```
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            # Push neighbors in reverse to maintain order
            for neighbor in reversed(graph[node]):
                stack.append(neighbor)

dfs_iterative(graph, 'A')
```

Time and Space Complexity

Let:

- V = number of vertices
- E = number of edges

Operation Complexity

Time $O(V + E)$

Space $O(V)$

☐ Applications of DFS

- ✓ Pathfinding in mazes
 - ✓ Topological Sorting (in DAGs)
 - ✓ Connected Components
 - ✓ Cycle Detection in Graphs
 - ✓ Solving puzzles (e.g., Sudoku)
 - ✓ Web Crawlers (Explore all linked pages)
-

☐ Real-world Analogy

Imagine a maze. DFS keeps going down one path until it can't go any further, then backtracks and tries another.
It's like exploring **every hallway deeply** before trying the next one.

🔍 What is Breadth-First Search (BFS)?

BFS is a graph traversal algorithm. It starts at a starting node (called the **root** in tree structures) and explores all the neighboring nodes at the current depth before moving on to the nodes at the next depth level.

🐘 **Think of it as:**

Waves of exploration — first explore all close nodes, then go deeper step by step.

✓ **It works for:**

- Graphs (connected or disconnected)
 - Trees (BFS in trees = Level Order Traversal)
-

☐ Core Concepts

∞ **Graph Terminology**

- **Node (or Vertex):** A point in the graph.
 - **Edge:** A connection between two nodes.
 - **Neighbor:** A node connected directly to another node.
-

☐ What BFS Needs:

- **Queue:** To process nodes in "First In First Out" (FIFO) order.
 - **Visited Set:** To keep track of already visited nodes (avoid infinite loops).
-

🔑 BFS Strategy (Step-by-step)

1. Start from the source node.
2. Mark it as visited.
3. Put it in a queue.
4. While the queue is not empty:
 - Remove the node from the front.
 - For each unvisited neighbor:

- Mark it as visited
- Add it to the queue

🔗 BFS Implementation in Python (from Scratch)

We'll represent the graph as an **adjacency list** using a dictionary.

Example Graph:

```
A -- B -- D
|      |
C      E
```

Adjacency list:

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A'],
    'D': ['B'],
    'E': ['B']
}
```

Python Code:

```
from collections import deque # for efficient queue

def bfs(graph, start_node):
    visited = set()           # To keep track of visited nodes
    queue = deque([start_node]) # Start with the starting node

    while queue:
        node = queue.popleft() # Remove node from the front of the queue

        if node not in visited:
            print("Visited:", node)
            visited.add(node)    # Mark the node as visited

            # Add all unvisited neighbors to the queue
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

# Run the BFS:
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A'],
    'D': ['B'],
    'E': ['B']
}

bfs(graph, 'A')
```

Output:

Visited: A
Visited: B
Visited: C
Visited: D
Visited: E

 How BFS Works (Trace)

- Start at A → Queue: ['B', 'C']
 - Visit B → Queue: ['C', 'D', 'E']
 - Visit C → Queue: ['D', 'E']
 - Visit D → Queue: ['E']
 - Visit E → Queue: [] (done!)
-

 When to Use BFS?

- ✓ Find **shortest path** in unweighted graphs
 - ✓ **Level order traversal** of trees
 - ✓ Web crawling
 - ✓ Social networking (shortest connection path)
 - ✓ Maze solving
-

 Key Points Summary

Concept	Description
Type	Traversal Algorithm
Data Structure	Queue (FIFO)
Time Complexity	$O(V + E) \rightarrow V$: Vertices, E : Edges
Space Complexity	$O(V)$ for visited set and queue
Use Case	Shortest path, Level order, Connectivity

Insertion Sort in Python – Detailed Explanation

◆ What is Insertion Sort?

Insertion Sort is a **simple sorting algorithm** inspired by how we arrange playing cards in our hand.

- Imagine picking cards one by one and inserting each card in its correct position relative to already sorted cards in your hand.

Steps:

1. Start from the **second element** (because a single element is already sorted).
2. Compare it with the elements before it.
3. Shift the elements greater than the current element to the right.
4. Insert the current element in the correct position.
5. Repeat until the list is sorted.

◆ Insertion Sort Algorithm in Python

```
def insertion_sort(arr):
    # Traverse from the second element to the end
    for i in range(1, len(arr)):
        key = arr[i]    # current element to insert
        j = i - 1       # index of the previous element

        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert the key in its correct position
        arr[j + 1] = key

    return arr

# Example usage
data = [12, 11, 13, 5, 6]
print("Original:", data)
print("Sorted:", insertion_sort(data))
```

✓ Output:

```
Original: [12, 11, 13, 5, 6]
Sorted: [5, 6, 11, 12, 13]
```

◆ Dry Run Example

Input: [12, 11, 13, 5, 6]

- Pass 1: [11, 12, 13, 5, 6]
 - Pass 2: [11, 12, 13, 5, 6] (already in correct place)
 - Pass 3: [5, 11, 12, 13, 6]
 - Pass 4: [5, 6, 11, 12, 13]
-

◆ Time Complexity of Insertion Sort

- **Best Case (Already Sorted):** $O(n)$
 - **Worst Case (Reverse Sorted):** $O(n^2)$
 - **Average Case:** $O(n^2)$
-

◆ Space Complexity

- $O(1) \rightarrow$ In-place sorting (uses only a few extra variables).
-

◆ Comparison with Other Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable?
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗ No (depends)
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ No

◆ Why use Insertion Sort?

- Simple and easy to implement.
- Efficient for **small arrays** or **partially sorted arrays**.
- Useful in real-time when new data keeps coming in (**online sorting**).
- Not suitable for large datasets because of **$O(n^2)$** time complexity.

Heap Sort – Explanation and Code

◆ What is Heap Sort?

Heap Sort is a **comparison-based sorting algorithm** that uses a **Binary Heap data structure** (usually a Max Heap) to sort elements.

It works in **two main steps**:

1. **Build a Max Heap** from the array.
 - A Max Heap is a complete binary tree where each node is greater than or equal to its children.
 - This ensures the **largest element is always at the root**.
2. **Extract elements from the heap one by one**:
 - Swap the root (largest element) with the last element in the heap.
 - Reduce the heap size by 1.
 - Heapify the root again to restore the heap property.
 - Repeat until the array is sorted.

◆ Why Heap Sort?

- **Time Complexity**:
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n \log n)$
- **Space Complexity**: $O(1)$ (in-place sorting).
- **Not a stable sort** (does not preserve the order of equal elements).

◆ The Heapify Process

Heapify is the **core operation** in Heap Sort.

Steps in Heapify:

1. Assume index i is the root.
2. Find indices of left child $= 2*i+1$ and right child $= 2*i+2$.
3. Compare root with its children.
4. Swap root with the largest child if needed.
5. Recursively heapify the affected subtree.

This ensures that the **subtree rooted at i satisfies the heap property**.

◆ Heap Sort Algorithm (Step by Step)

1. **Build a Max Heap** from the given array (call heapify for all non-leaf nodes).
 2. **Swap the root with the last element.**
 3. **Heapify the reduced heap.**
 4. **Repeat** until the entire array is sorted.
-

◆ Python Code for Heap Sort

```
def heapify(arr, n, i):
    """Ensure the subtree rooted at index i is a max heap."""
    largest = i          # Assume root is largest
    left = 2 * i + 1     # left child
    right = 2 * i + 2    # right child

    # If left child exists and is greater than root
    if left < n and arr[left] > arr[largest]:
        largest = left

    # If right child exists and is greater than largest so far
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If root is not largest, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest) # Recursive heapify

def heap_sort(arr):
    n = len(arr)

    # Step 1: Build a max heap
    for i in range(n // 2 - 1, -1, -1): # Start from last non-leaf node
        heapify(arr, n, i)

    # Step 2: Extract elements from heap one by one
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0] # Swap root with last element
        heapify(arr, i, 0)              # Heapify reduced heap

# Example usage
arr = [12, 11, 13, 5, 6, 7]
print("Original array:", arr)
heap_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

```
arr = [12, 11, 13, 5, 6, 7]
```

1. **Build Max Heap**
→ [13, 11, 12, 5, 6, 7]
 2. **Extract 1st max (13)**
 - Swap root with last: [7, 11, 12, 5, 6, 13]
 - Heapify → [12, 11, 7, 5, 6, 13]
 3. **Extract 2nd max (12)**
 - Swap: [6, 11, 7, 5, 12, 13]
 - Heapify → [11, 6, 7, 5, 12, 13]
 4. Continue the process...
Final sorted array → [5, 6, 7, 11, 12, 13].
-

◆ Output

Original array: [12, 11, 13, 5, 6, 7]
Sorted array: [5, 6, 7, 11, 12, 13]

◆ Summary

- Heap Sort uses a **binary heap** to sort data.
- It is an **in-place sorting algorithm** with $O(1)$ extra space.
- It always runs in $O(n \log n)$ time, regardless of input.
- It is **not stable**, but it is **efficient** and widely used.

Selection Sort – Explanation and Code

◆ What is Selection Sort?

Selection Sort is one of the simplest **comparison-based sorting algorithms**.

It works by **repeatedly finding the smallest element** from the unsorted part of the array and moving it to the beginning.

◆ How Selection Sort Works (Step by Step)

1. Start with the first element (at index 0).
 2. Find the **minimum element** from the unsorted part of the array.
 3. Swap the minimum element with the element at the beginning of the unsorted part.
 4. Move the boundary of the sorted part forward by one.
 5. Repeat until the whole array is sorted.
-

◆ Why Selection Sort?

- **Time Complexity:**
 - Best Case: $O(n^2)$
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$
 - **Space Complexity:** $O(1)$ (in-place sorting).
 - **Not stable** (equal elements may lose their relative order).
 - **Very easy to understand**, but inefficient for large datasets compared to Heap Sort, Merge Sort, or Quick Sort.
-

◆ Selection Sort Algorithm

1. Loop i from 0 to $n-1$.
 2. Assume i is the index of the minimum element.
 3. Compare with every other element in the unsorted part.
 4. Update index of the minimum if a smaller element is found.
 5. Swap the found minimum with element at index i .
-

◆ Python Code for Selection Sort

```
def selection_sort(arr):
    n = len(arr)

    # Traverse through all elements
    for i in range(n):
        # Assume the minimum is the first element in the unsorted part
        min_index = i

        # Find the index of the minimum element in the unsorted part
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the found minimum with the first element of the unsorted
part
        arr[i], arr[min_index] = arr[min_index], arr[i]

# Example usage
arr = [64, 25, 12, 22, 11]
print("Original array:", arr)
selection_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[64, 25, 12, 22, 11]

1. **Pass 1:**
 - Minimum element = 11
 - Swap with first element → [11, 25, 12, 22, 64]
 2. **Pass 2:**
 - Minimum element = 12
 - Swap with second element → [11, 12, 25, 22, 64]
 3. **Pass 3:**
 - Minimum element = 22
 - Swap with third element → [11, 12, 22, 25, 64]
 4. **Pass 4:**
 - Minimum element = 25
 - Swap with fourth element → [11, 12, 22, 25, 64]
 5. **Pass 5:**
 - Last element already in place → [11, 12, 22, 25, 64]
-

◆ Output

Original array: [64, 25, 12, 22, 11]
Sorted array: [11, 12, 22, 25, 64]

◆ Summary

- Selection Sort repeatedly finds the minimum element and puts it in the correct position.
- It has **$O(n^2)$** time complexity regardless of input.
- It is **in-place** but **not stable**.
- Useful for **small datasets** or when simplicity is more important than performance.

Merge Sort – Explanation and Code

◆ What is Merge Sort?

Merge Sort is a **divide and conquer algorithm** used for sorting.

It works by **dividing the array into smaller sub-arrays**, sorting them recursively, and then **merging the sorted sub-arrays** to produce the final sorted array.

◆ How Merge Sort Works (Step by Step)

1. **Divide:** Split the array into two halves (left and right).
 2. **Conquer:** Recursively sort the two halves.
 3. **Combine:** Merge the two sorted halves into a single sorted array.
-

◆ Why Merge Sort?

- **Time Complexity:**
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n \log n)$
 - **Space Complexity:** $O(n)$ (requires extra memory for merging).
 - **Stable sort** (preserves the order of equal elements).
 - **Efficient for large datasets**, unlike Selection or Bubble Sort.
-

◆ Merge Sort Algorithm

1. If the array has **0 or 1 element**, it is already sorted.
 2. Otherwise:
 - Divide the array into two halves.
 - Recursively apply merge sort on both halves.
 - Merge the two sorted halves together.
-

◆ Python Code for Merge Sort

```
def merge_sort(arr):
    if len(arr) > 1:
        # Find the middle point
        mid = len(arr) // 2

        # Divide the array into two halves
        left_half = arr[:mid]
        right_half = arr[mid:]

        # Recursively sort both halves
        merge_sort(left_half)
        merge_sort(right_half)

        # Merge the two halves
        i = j = k = 0

        # Compare elements from both halves and merge
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
```

```
# Copy remaining elements of left_half, if any
while i < len(left_half):
    arr[k] = left_half[i]
    i += 1
    k += 1

# Copy remaining elements of right_half, if any
while j < len(right_half):
    arr[k] = right_half[j]
    j += 1
    k += 1

# Example usage
arr = [38, 27, 43, 3, 9, 82, 10]
print("Original array:", arr)
merge_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[38, 27, 43, 3, 9, 82, 10]

1. **Divide Step:**
 - Split into [38, 27, 43] and [3, 9, 82, 10].
 - Keep splitting until single elements remain.
 2. **Sorting Single Elements (Base Case):**
 - [38] [27] [43] [3] [9] [82] [10]
 3. **Merge Step:**
 - Merge [38] and [27] → [27, 38]
 - Merge [27, 38] and [43] → [27, 38, 43]
 - Merge [3] and [9] → [3, 9]
 - Merge [3, 9] and [82] → [3, 9, 82]
 - Merge [3, 9, 82] and [10] → [3, 9, 10, 82]
 4. **Final Merge:**
 - Merge [27, 38, 43] and [3, 9, 10, 82] → [3, 9, 10, 27, 38, 43, 82]
-

◆ Output

Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array: [3, 9, 10, 27, 38, 43, 82]

◆ Summary

- Merge Sort uses **divide and conquer** to sort arrays.

- Runs in $O(n \log n)$ time for all cases.
 - Requires $O(n)$ extra space.
 - It is a **stable sort**, making it useful in scenarios where the relative order of equal elements matters.
 - Works well for **large datasets** and **linked lists**.
-

Quick Sort – Explanation and Code

◆ What is Quick Sort?

Quick Sort is a **divide and conquer sorting algorithm** that works by selecting a **pivot element**, partitioning the array around the pivot, and then recursively sorting the sub-arrays.

It is one of the fastest sorting algorithms in practice and is widely used in real-world applications.

◆ How Quick Sort Works (Step by Step)

1. **Choose a Pivot:** Pick an element as the pivot (commonly the last element, but other strategies exist).
 2. **Partition:** Rearrange elements so that:
 - Elements **smaller than the pivot** go to the left.
 - Elements **greater than the pivot** go to the right.
 3. **Recursively Apply:** Apply Quick Sort on the left and right sub-arrays.
 4. **Combine:** Since sorting is done in place, the array becomes sorted when recursion ends.
-

◆ Why Quick Sort?

- **Time Complexity:**
 - Best Case: $O(n \log n)$ (balanced partitions).
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n^2)$ (if pivot choice is poor, e.g., sorted array with last element as pivot).
 - **Space Complexity:** $O(\log n)$ (due to recursion stack).
 - **Not stable** (relative order of equal elements may change).
 - **Very efficient in practice** because of good cache performance and fewer comparisons.
-

◆ Quick Sort Algorithm

1. Pick a pivot element.

2. Partition the array into two sub-arrays: elements smaller than pivot, elements larger than pivot.
 3. Recursively apply Quick Sort on the sub-arrays.
 4. Combine results (in-place).
-

◆ Python Code for Quick Sort

```
def partition(arr, low, high):
    # Choose the last element as pivot
    pivot = arr[high]
    i = low - 1 # index of smaller element

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i] # swap if element <= pivot

    # Place pivot at the correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        # Partition index
        pi = partition(arr, low, high)

        # Recursively sort elements before and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# Example usage
arr = [10, 7, 8, 9, 1, 5]
print("Original array:", arr)
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[10, 7, 8, 9, 1, 5]

1. **First Partition:**
 - Pivot = 5
 - Rearranged → [1, 5, 8, 9, 10, 7]
 - Pivot 5 is placed in correct position (index 1).
2. **Left Sub-array:** [1] → already sorted.
3. **Right Sub-array:** [8, 9, 10, 7]
 - Pivot = 7

- Rearranged $\rightarrow [7, 9, 10, 8]$
 - Pivot 7 at correct position.
4. Continue partitioning until all sub-arrays are of size 1.

Final sorted array $\rightarrow [1, 5, 7, 8, 9, 10]$.

◆ Output

Original array: [10, 7, 8, 9, 1, 5]
Sorted array: [1, 5, 7, 8, 9, 10]

◆ Summary

- Quick Sort uses **divide and conquer** with partitioning around a pivot.
 - Average case runs in $O(n \log n)$, but worst case is $O(n^2)$.
 - Performs better than Merge Sort in practice (due to lower memory overhead).
 - **In-place** but **not stable**.
 - One of the **most widely used sorting algorithms** in libraries and systems.
-

Counting Sort – Explanation and Code

◆ What is Counting Sort?

Counting Sort is a **non-comparison-based sorting algorithm**.

Instead of comparing elements (like Quick Sort or Merge Sort), it works by **counting the number of occurrences** of each element in the input.

It is mainly useful when:

- The input elements are **integers**.
 - The range of elements ($\text{max} - \text{min}$) is **not significantly larger than the number of elements**.
-

◆ How Counting Sort Works (Step by Step)

1. **Find the maximum element** in the array (let's call it k).
 2. **Create a count array** of size $k+1$, initialized with zeros.
 3. **Count each element's frequency** by iterating through the input array.
 4. **Update count array** to store cumulative counts (this tells us positions of elements in the sorted array).
 5. **Build the output array** by placing elements in their correct positions using the count array.
 6. Copy the output array back to the original array.
-

◆ Why Counting Sort?

- **Time Complexity:**
 - Best Case: $O(n + k)$
 - Average Case: $O(n + k)$
 - Worst Case: $O(n + k)$
 - **Space Complexity:** $O(n + k)$ (extra space needed for count and output arrays).
 - **Stable sort** (if implemented properly).
 - **Efficient for small ranges**, but **not suitable for large ranges** (e.g., sorting numbers up to billions).
-

◆ Counting Sort Algorithm

1. Find the maximum element in the array.
 2. Initialize a count array of that size.
 3. Count occurrences of each element.
 4. Compute cumulative sums to determine positions.
 5. Build the output array.
 6. Copy it back to the original.
-

◆ Python Code for Counting Sort

```
def counting_sort(arr):
    # Step 1: Find the maximum element
    max_val = max(arr)

    # Step 2: Initialize count array
    count = [0] * (max_val + 1)

    # Step 3: Count the occurrences of each element
    for num in arr:
        count[num] += 1

    # Step 4: Update count[i] to store cumulative sum
    for i in range(1, len(count)):
        count[i] += count[i - 1]

    # Step 5: Build the output array (stable)
    output = [0] * len(arr)
    for num in reversed(arr): # reversed to maintain stability
        output[count[num] - 1] = num
        count[num] -= 1

    # Step 6: Copy sorted elements back into original array
    for i in range(len(arr)):
        arr[i] = output[i]

# Example usage
arr = [4, 2, 2, 8, 3, 3, 1]
print("Original array:", arr)
counting_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[4, 2, 2, 8, 3, 3, 1]

1. **Find max element:** 8
→ Create count array of size 9 → [0, 0, 0, 0, 0, 0, 0, 0, 0]

2. **Count occurrences:**

After counting $\rightarrow [0, 1, 2, 2, 1, 0, 0, 0, 1]$
(meaning: one 1, two 2s, two 3s, one 4, one 8)

3. **Cumulative sum:**

$\rightarrow [0, 1, 3, 5, 6, 6, 6, 6, 7]$
(now each index tells us the final position of that number).

4. **Build output (placing elements in correct order):**

$\rightarrow [1, 2, 2, 3, 3, 4, 8]$

◆ Output

Original array: `[4, 2, 2, 8, 3, 3, 1]`
Sorted array: `[1, 2, 2, 3, 3, 4, 8]`

◆ Summary

- Counting Sort is a **non-comparison sorting algorithm**.
 - Runs in $O(n + k)$ time, where k is the range of input values.
 - Requires extra memory ($O(n + k)$), so it is **not memory efficient for large ranges**.
 - **Stable sort**, which makes it suitable for sorting objects by multiple keys.
 - Best for integers with a **small range** relative to the array size.
-

Bubble Sort – Explanation and Code

◆ What is Bubble Sort?

Bubble Sort is the **simplest comparison-based sorting algorithm**.

It repeatedly **swaps adjacent elements** if they are in the wrong order. After each pass, the largest element "bubbles up" to its correct position at the end of the array.

◆ How Bubble Sort Works (Step by Step)

1. Start from the beginning of the array.
 2. Compare the first two adjacent elements.
 - If the first is greater than the second → swap them.
 3. Move to the next pair, and repeat until the end of the array.
 4. After the first pass, the **largest element is at the end**.
 5. Repeat the process for the remaining elements (ignoring the last sorted ones).
 6. Continue until no swaps are needed → array is sorted.
-

◆ Why Bubble Sort?

- **Time Complexity:**
 - Best Case: $O(n)$ (if array already sorted, with optimization).
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$
 - **Space Complexity:** $O(1)$ (in-place sorting).
 - **Stable sort** (preserves the order of equal elements).
 - **Easy to understand**, but very inefficient for large datasets.
-

◆ Bubble Sort Algorithm

1. Loop i from 0 to $n-1$.
 2. For each pass, loop j from 0 to $n-i-1$.
 3. Compare adjacent elements `arr[j]` and `arr[j+1]`.
 4. Swap if they are in the wrong order.
 5. Stop early if no swaps happen in a pass (optimization).
-

◆ Python Code for Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False # Optimization: check if any swap happens

        # Last i elements are already in place
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swaps happened → array is already sorted
        if not swapped:
            break

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
print("Original array:", arr)
bubble_sort(arr)
print("Sorted array:", arr)
```

◆ Example Walkthrough

Take the array:

[64, 34, 25, 12, 22, 11, 90]

1. **Pass 1:**
After comparing/swapping → [34, 25, 12, 22, 11, 64, 90]
(largest 90 is bubbled to the end).
2. **Pass 2:**
→ [25, 12, 22, 11, 34, 64, 90]
3. **Pass 3:**
→ [12, 22, 11, 25, 34, 64, 90]
4. **Pass 4:**
→ [12, 11, 22, 25, 34, 64, 90]
5. **Pass 5:**
→ [11, 12, 22, 25, 34, 64, 90]

Array is now sorted.

◆ Output

Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]

◆ Summary

- Bubble Sort works by **repeatedly swapping adjacent elements** until the array is sorted.
 - It is **stable** and **in-place**, but very inefficient ($O(n^2)$).
 - Useful only for **small datasets or teaching purposes**.
 - Rarely used in practice compared to **Merge Sort, Quick Sort, or Heap Sort**.
-